

Towards Lightweight CKKS: On Client Cost Efficiency

Jung Hee Cheon
Seoul National University
Seoul, Republic of Korea
CryptoLab Inc.
Seoul, Republic of Korea
jhcheon@snu.ac.kr

Minsik Kang
Seoul National University
Seoul, Republic of Korea
kaiser351@snu.ac.kr

Jai Hyun Park
CryptoLab Inc.
Lyon, France
jaihyunp@gmail.com

ABSTRACT

Fully homomorphic encryption (FHE) enables clients with small devices to securely delegate their computations to powerful servers. However, to delegate these computations, a client should generate and transmit several gigabytes of FHE keys to the server. Reducing the size of FHE keys without compromising efficiency is therefore highly desirable, particularly for applications involving mobile and IoT devices.

In this work, we propose two new key management systems, KG^+ and BTS^+ , which reduce the client-side cost of FHE. The KG^+ system significantly reduces the key size without any compromise in the efficiency of homomorphic computation compared to the state-of-the-art key management system by Lee-Lee-Kim-No [Asiacrypt 2023]. The BTS^+ system further reduces the key size at the expense of granularity in homomorphic computation.

Our systems rely on a new ring switching technique for keys that bridges FHE keys with different parameters. This technique decouples the rings of FHE keys during transmission and computation. In our approach, the client generates and sends “transmission keys” with size-optimal parameters, and the server generates “evaluation keys” with computation-optimal parameters.

We evaluate the communication costs of KG^+ and BTS^+ on the tasks studied in Lee-Lee-Kim-No. The transmission key sizes for CKKS bootstrapping with ring dimension 2^{16} are 325–609 MB for KG^+ and 285 MB for BTS^+ . These are 3.09–4.37 $\times$ and 3.51–9.30 $\times$ smaller than Lee-Lee-Kim-No, respectively. For the secure ResNet-20 inference, the key sizes for KG^+ and BTS^+ are 325–609 MB and 285 MB, respectively. These are 3.95–5.73 $\times$ and 4.53–12.25 $\times$ smaller than Lee-Lee-Kim-No.

KEYWORDS

Homomorphic Encryption, Transmission Keys, Hierarchical Rotation Keys

1 INTRODUCTION

The large key size of homomorphic encryption (HE) has been a bottleneck in real-world applications on small devices. Many applications of HE for privacy-preserving machine learning (PPML) rely on ring learning with error problem [23, 26] (RLWE) to achieve high throughput. However, RLWE-based HE schemes [7, 8, 10, 16] often require tens of gigabytes of evaluation keys, leading to severe computation and communication costs and memory overhead to generate, send, and store the keys. For example, in [22], the authors pointed out that for the secure inference of ResNet-20, the overall evaluation key size is often more than 30 GB.

In PPML applications, the large key size is a problem for the clients, who delegate their computations to servers using HE. Note that a client usually has significantly weaker computational power than the computing server; for example, clients might use mobile devices. Generating and transmitting gigabytes of keys and ciphertexts is not desirable for clients. The setup cost for the clients should be low, at least lower than the cost of machine learning task itself.

One can reduce the key size by adopting either LWE-based HE schemes [11, 15], RLWE-based HE with small parameters [4], or RLWE-based HE with only a few evaluation keys. However, all the above solutions result in inefficient homomorphic computation in terms of throughput. Low throughput is not appropriate for most PPML applications with large machine learning models. In this work, we focus on reducing the key size for HE with sufficiently high throughputs.

Surprisingly, there have been only a few works addressing the large HE keys for RLWE-based HE schemes with high throughputs. The most relevant prior work is [22], which introduced a *hierarchical key system* to address the heavy client’s cost of the HE key generation. In their system, a client sends a small set of transmission keys to the server, and the server generates many evaluation keys from the received transmission keys. This approach charges the server an extra cost, but it substantially reduces clients’ costs which is the main concern in many HE applications.

However, the results in [22] are not sufficient for many real applications. It still requires sending many transmission keys with large parameters to support efficient online homomorphic computation. For example, for the secure inference of ResNet-20, [22] requires 2.67 GB of transmission keys. While this is a significant improvement over the previous key size of 54.3 GB in [21], 2.67 GB still is considerably large, particularly when clients use mobile phones or IoT devices.

1.1 Contributions

We introduce new key management systems, KG^+ and BTS^+ , which reduce the size of transmission keys for RLWE-based HE with high throughputs. KG^+ reduces the key size without compromising in homomorphic computation, while the BTS^+ system further reduces the key size with compromising the granularity of homomorphic computation.

The first key management system KG^+ provides a smaller transmission key size than the state-of-the-art [22] while providing equally efficient homomorphic computation after the server-side key generation. In [22], a client sends several large transmission keys. In KG^+ , the client sends a few small transmission keys with size-optimal parameters along with a single large key that we call

	[21]	[22]	[22] (optimized)	KG ⁺ (ours)	BTS ⁺ (ours)
Comm. Cost	38.3	8.31	3.49	0.609	0.285

Table 1: Communication cost (in gigabytes) for HE keys in secure ResNet-20 inference under A.i parameter set (Section 5). For [22], we report both the original result from the paper (without known optimizations) and our implementation with known optimization techniques.

the “homing key”. From the received transmission keys, the server can generate HE keys with the efficiency-optimal parameters.

The second key management system BTS⁺ further reduces the size of the homing key. In BTS⁺, a client sends a smaller homing key along with more transmission keys, of which the overall size is still small. We note that BTS⁺ might compromise the granularity of the homomorphic computation after the key generation. However, the throughput of the computation would be nearly the same.

In both systems, we generate the evaluation keys in the extension ring of the underlying ring of homomorphic computations. By using the extension ring, the server could have sufficient moduli budget for the key generation, and it allows the smaller transmission keys as inputs. We introduce several tools for the key generation in the extension ring. We extend the ring switching technique [17] for the HE keys. Then, we compress multiple HE keys into a single ciphertext in the extension ring. To put it all together with the hierarchical key management system [22], we propose KG⁺ and BTS⁺.

We provide CPU and GPU implementations of KG⁺. In addition, we estimate the key size of KG⁺ and BTS⁺ for the parameters in [22]. It shows that KG⁺ reduces the client key size to 325–609 MB, while BTS⁺ further reduces it to 285 MB across different HE parameter sets. This corresponds to 3.95–5.73× and 4.53–12.25× improvements compared to the optimized approach in [22], respectively.

1.2 Technical Overview

Before diving into details, we introduce the existing attempt to reduce transmission key size and discuss its limitations. Let $N \geq 2$ be a power of two. We denote the cyclotomic ring $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and let $\mathcal{R}' = \mathbb{Z}[Y]/(Y^{2N} + 1)$. We regard $\mathcal{R}$ as a proper subring of $\mathcal{R}'$ via the embedding $X \mapsto Y^2$.

Hierarchical key system in [22]. Our starting point is the hierarchical key system from [22]. HE often requires a large number of large evaluation keys. The paper addresses the “large number” of evaluation keys. The authors suggest sending a small number of master rotation keys and generating many rotation keys from the master keys. We refer to Section 2.4 for the details.

In the existing hierarchical key system, each master rotation key was large. Note that RLWE-based HE schemes have a limited moduli budget based on the fixed security level and degree of the underlying ring. To avoid the degradation of homomorphic computation, one should not consume many moduli while generating keys in the hierarchical system. This is only possible if each master rotation key is sufficiently large.

For instance, in the above examples, PQ (the modulus of rotation keys) is initially set to close to a certain moduli budget, $Q_{\max}$. At the same time, $P^{(1)}PQ$ (the modulus of master rotation keys) should be less than $Q_{\max}$ for the same level of security as it underlies in the same ring. Consequently, the size of $P^{(1)}$ is much smaller than

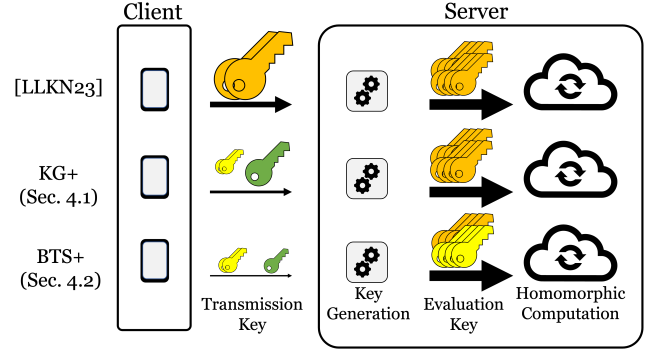


Figure 1: Visualization of the key management systems: [22], KG⁺, and BTS⁺. See Sections 4.1 and 4.2 for the detailed description of our key management systems.

PQ . Moreover, the “gadget rank” of master rotation keys is large, which results in the large size of each master rotation key.

Our approach. In this paper, we propose two key management systems: KG⁺ and BTS⁺. In contrast to [22], they allow for small-size transmission keys. Figure 1 illustrates the new key management systems. For this purpose, we use different parameters for the transmission keys and the evaluation keys. Precisely, for transmission keys, we utilize the extension ring to generate the keys using a rich moduli budget: $P^{(1)}PQ$ is set to be larger than $Q_{\max}$ by using a larger ring degree. This allows one to use small gadget ranks for transmission keys, which significantly reduces their size.

In order to adopt the extension ring for the key generation, we consider the intermediate key in the extension ring during the key generation. The intermediate keys belong to the extension ring, but they contain the information about the secret in the original ring. From the given transmission keys in an extension ring, we generate the intermediate key and switch the underlying ring of the intermediate keys to obtain the desired evaluation keys in the subring.

Below, we describe in detail our three techniques: (1) ring switching for keys, (2) rotation key generation over the extension ring, and (3) general switching key generation over the extension ring.

Ring switching for keys. The main tool we suggest is ring-switching for keys. In the literature, the ring-switching technique is for the ciphertexts rather than keys. We extend the ring-switching technique [8, 17] to switch the ring of HE keys. The modified ring-switching technique switches the underlying ring of HE keys from $\mathcal{R}'$ to $\mathcal{R}$.

We start from the observation that a HE key is a tuple of ciphertexts. In a nutshell, a HE key can be written as:

$$\{\text{Enc}_s(PG_i \cdot s')\}_{i=0, \dots, \text{dnum}-1},$$

where P and G_i ’s are constants. Then, we switch its underlying ring of the key by applying ring switching to the encryption $\text{Enc}_s(PG_i \cdot$

s') for each i . It outputs the encryption $\text{Enc}_{\tilde{s}}(PG_i \cdot s')$ under distinct $\tilde{s}$ over another ring, which completes the ring-switching for the HE key.

To the best of our knowledge, this is the first time to utilize the ring-switching technique [17] for the keys rather than ciphertexts. We note that ring-switching for keys requires a ring-switching key, which we call a “homing key”.

Rotation key generation over an extension ring. Suppose that we are given a ring $\mathcal{R}$ and its extension ring $\mathcal{R}'$. To generate the rotation keys in $\mathcal{R}$ from the transmission keys in $\mathcal{R}'$, we modify the hierarchical key system from [22]. Each rotation key rotk_r in $\mathcal{R}$ is related to $\text{Enc}_s(s(X^{5^r}))$, in which s is a secret key in $\mathcal{R}$. In the extension ring $\mathcal{R}'$, we consider its intermediate key, which is $\text{Enc}_{s'}(s(X^{5^r}))$, where s' is a secret key in $\mathcal{R}'$.

We can generate many intermediate keys in $\mathcal{R}'$ from a HE key $\text{Enc}_{s'}(s(X))$, and master rotation keys in higher modulus. For example, we can generate $\text{Enc}_{s'}(s(X^{5^r}))$ from $\text{Enc}_{s'}(s(X))$ with r key-switching operations using $\text{Enc}_{s'}(s'(X^5))$. This is our new hierarchical key system with an extension ring: generate intermediate keys from the master rotation keys in the extension ring. We note that our new hierarchical key system can easily adopt the multi-level approach as in [22]. After generating all intermediate rotate keys, we apply ring-switching to obtain the final rotate keys in $\mathcal{R}$. We refer to Section 3.3 for more details.

General key generation over an extension ring. The hierarchical key system in [22] does not consider reducing the general types of HE keys other than rotation keys. In practice, one often needs to transfer other evaluation keys together, such as a multiplication key, conjugation key, and general key-switching keys. We describe how to utilize the ring-switching technique to reduce the general types of HE keys.

Again, we use the fact that a HE key is a tuple of ciphertexts. Importantly, we can *multiply the HE keys by a constant*. As $\mathcal{R}'$ provides a larger moduli budget, we can multiply the HE keys in $\mathcal{R}'$ with high precision. We utilize it for key generation as follows. A key-switching key from s_0 to s_1 in $\mathcal{R}$ is a form of:

$$\text{swk}_{s_0 \rightarrow s_1} = \{\text{Enc}_{s_1}(PG_i \cdot s_0)\}_{i=0, \dots, \text{dnum}-1}.$$

We consider an intermediate key in $\mathcal{R}'$ as

$$\text{IntSwk}_{s_0 \rightarrow s'} = \{\text{Enc}_{s'}(PG_i \cdot s_0)\}_{i=0, \dots, \text{dnum}-1},$$

where s' is a secret key in $\mathcal{R}'$. Then, suppose we are given a transmission key

$$\text{Enc}_{s'}(PP' \cdot s_0),$$

in which P' is a special modulus with $P' \approx G_i$ for all i 's. By multiplying by a constant G_i/P' , we can generate the intermediate key from the transmission key. Finally, the ring-switching technique converts IntSwk to swk , and it completes the key generation. Compared to $\text{swk}_{s_{k_0} \rightarrow s_1}$, $\text{Enc}_{s'}(PP' \cdot s_0)$ is relatively small unless dnum is too small.

KG⁺ key management system. To put it all together, we propose KG⁺, a new key management system. In KG⁺, a user sends a homing key in $\mathcal{R}$ and a few transmission keys in $\mathcal{R}'$. The server then generates *intermediate keys* in $\mathcal{R}'$ from the transmission keys, and it switches the ring of the intermediate keys from $\mathcal{R}'$ to $\mathcal{R}$ by using the homing key. It ends up with all required evaluation keys in

$\mathcal{R}$, and the server can perform homomorphic computations using them.

KG⁺ reduces transmission key size than [22]. KG⁺ sends an even smaller number of transmission keys than [22], and each transmission key except the homing key is smaller than [22].

BTS⁺ key management system. In KG⁺, the homing key belongs to $\mathcal{R}$, which has a tight moduli budget. Consequently, the gadget rank of the homing key, $\text{dnum}^{(h)}$, should be sufficiently large, and the size of the homing key is large.

We suggest another key management system, BTS⁺, to downsize the homing key. Our new system utilizes the extension ring for both key generation and a part of homomorphic computation (e.g., bootstrapping). By doing this, the homing key can have a small size with a small gadget rank. To be more precise, we set a modulus Q_{mid} in BTS⁺, and for the ciphertexts with moduli larger (resp. smaller) than Q_{mid} , we use $\mathcal{R}'$ (resp. $\mathcal{R}$) for the homomorphic computation. For the computation over moduli larger than Q_{mid} , as we use $\mathcal{R}'$, the efficiency of homomorphic computation might be degraded. Nevertheless, its throughput would be similar, and once there is more than one ciphertext, the efficiency degradation would not be significant. We refer to Section 4.2 for more details.

Compatibility with RLWE-based HE schemes. While the main focus is on CKKS scheme, the proposed methods are compatible with other RLWE-based HE schemes, such as BGV [8] and BFV [16]. When using power-of-two cyclotomic rings, which is the standard ring choice in current FHE libraries [1, 2, 13], all methods can be directly applied. Note that the proposed key generation does not exploit any scheme-specific plaintext encoding structure.

Furthermore, using a proper extension ring and transmission keys, the proposed methods can also be applied to non-power-of-two cyclotomic rings [18]. In particular, it only requires (1) a degree-2 ring extension and (2) a set of transmission keys that can generate all automorphism keys. Since the automorphism group for a general cyclotomic ring $\mathcal{R} = \mathbb{Z}[X]/(\Phi_M(X))$ with non-power-of-two M could be different from that for CKKS, the set of generators and transmission keys should be adapted accordingly. The choice of extension ring should also be modified as follows: if M is even, we set $\mathcal{R}' = \mathbb{Z}[Y]/(\Phi_{2M}(Y))$ and embed $\mathcal{R}$ into $\mathcal{R}'$ via $X \mapsto Y^2$; otherwise, we set $\mathcal{R}' = \mathbb{Z}[Y]/(\Phi_{4M}(Y))$ and embed via $X \mapsto -Y^2$. In both cases¹, the extension degree is $[\mathcal{R}' : \mathcal{R}] = 2$.

In the remainder of this paper, we instantiate our key management system, focusing mainly on the CKKS scheme over standard power-of-two cyclotomic rings.

2 PRELIMINARY

For a given power-of-two integer $N \geq 2$, we denote the cyclotomic polynomial ring $\mathcal{R}_N$ as $\mathcal{R}_N = \mathbb{Z}[X]/(X^N + 1)$. The residue ring modulo an integer Q is denoted by $\mathcal{R}_{N,Q} = \mathcal{R}_N/Q\mathcal{R}_N$. When the ring dimension N is fixed, we often simply denote $\mathcal{R}_N$ as $\mathcal{R}$ and let $\mathcal{R}' = \mathcal{R}_{2N} = \mathbb{Z}[Y]/(Y^{2N} + 1)$. We note that $\mathcal{R}$ is a proper subring of $\mathcal{R}'$ via an embedding map $X \mapsto Y^2$.

We use $x \leftarrow D$ to denote the sampling x according to a distribution D . For a ring degree N , we define some distributions over

¹If M is even, then $\Phi_{2M}(Y) = \Phi_M(Y^2)$ and $\varphi(2M) = 2\varphi(M)$; if M is odd, then $\Phi_{4M}(Y) = \Phi_M(-Y^2)$ and $\varphi(4M) = 2\varphi(M)$.

$\mathcal{R}$: χ_h with coefficients uniformly distributed over $\{-1, 0, 1\}$ and exactly h non-zero coefficients, and χ_σ with coefficients sampled independently from the discrete Gaussian distribution of variance σ^2 . For a positive integer n , we denote a set of non-negative integers smaller than n as $[n]$, i.e., $[n] = \{0, 1, \dots, n-1\}$. A RLWE-based ciphertext of a plaintext $m \in \mathcal{R}$ is a pair $(b, a) \in \mathcal{R}_Q^2$ satisfying:

$$b = -a \cdot s + m + e \pmod{Q},$$

where $s \leftarrow \chi_h$ is a secret, Q is the ciphertext modulus, $a \leftarrow \mathcal{R}_Q$, and $e \leftarrow \chi_\sigma$ is error.

2.1 RNS-based Gadget Decomposition and Key Switching

For a ciphertext modulus $Q = \prod_{i=0}^{\ell-1} q_i$, which is a product of pair-wise coprimes $q_0, \dots, q_{\ell-1}$, we introduce a RNS-based gadget decomposition of rank dnum . Let $\alpha = \lceil \ell / \text{dnum} \rceil$ and $Q_i = \prod_{j=i\alpha}^{(i+1)\alpha-1} q_j$ for $0 \leq i < \text{dnum}$. The RNS-based gadget decomposition introduced in [19] is $h: \mathcal{R}_Q \rightarrow \mathcal{R}^{\text{dnum}}$ which maps a into $([\hat{Q}_0^{-1} \cdot a]_{Q_0}, \dots, [\hat{Q}_{\text{dnum}-1}^{-1} \cdot a]_{Q_{\text{dnum}-1}})$ where $\hat{Q}_i = Q/Q_i$. We call $\mathbf{G} = (G_0, \dots, G_{\text{dnum}-1}) = (\hat{Q}_0, \dots, \hat{Q}_{\text{dnum}-1}) \in \mathcal{R}_Q^{\text{dnum}}$. Then it follows that

$$\langle h(a), \mathbf{G} \rangle = \sum_{i=0}^{\text{dnum}-1} [a \cdot \hat{Q}_i^{-1}]_{Q_i} \cdot \hat{Q}_i = a \pmod{Q}.$$

The gadget decomposition is useful for the key-switching operation in the RNS-CKKS scheme along with a special modulus P .

- $\text{SwkGen}(N, Q, QP, s', s)$: For given secret keys $s, s' \in \mathcal{R}_N$, it outputs $\text{swk} = \{(\beta_i, \alpha_i)\}_{i \in [\text{dnum}]} \in \mathcal{R}_{N, QP}^{2 \times \text{dnum}}$ such that for each i ,

$$\beta_i = -\alpha_i \cdot s + PG_i \cdot s' + e_i \pmod{QP},$$

where $\alpha_i \leftarrow \mathcal{R}_{N, QP}$, and $e_i \leftarrow \chi_{N, \sigma}$. Here, one can write $(\beta_i, \alpha_i) = \text{Enc}_s(PG_i \cdot s')$.

- $\text{KeySwitch}_{\text{swk}}(\text{ct})$: For $\text{swk} = (\text{swk}_0, \text{swk}_1) \in \mathcal{R}_{N, Q}^{2 \times \text{dnum}}$ and $\text{ct} = (b, a) \in \mathcal{R}_{N, Q}^2$, it outputs

$$\left(b + \left\lceil \frac{\langle h(a), \text{swk}_0 \rangle}{P} \right\rceil, \left\lceil \frac{\langle h(a), \text{swk}_1 \rangle}{P} \right\rceil \right) \in \mathcal{R}_{N, Q}^2.$$

We note that P is selected to satisfy $P \geq \max_i Q_i$ to ensure the valid key-switching operation. Once we choose the prime moduli $q_0, q_1, \dots, q_{\ell}$ of the same size, it suffices to choose P as $P \geq Q^{1/\text{dnum}}$.

We can rotate, conjugate, and multiply the ciphertexts using key-switchings. Each operation requires a key-switching key.

$$\text{mul}_k \leftarrow \text{SwkGen}(N, Q, QP, s^2, s),$$

$$\text{rot}_{k_r} \leftarrow \text{SwkGen}(N, Q, QP, s(X^{5^r}), s),$$

$$\text{cjk} \leftarrow \text{SwkGen}(N, Q, QP, s(X^{-1}), s).$$

Then, one can multiply, rotate, and conjugate the ciphertexts as follows.

- $\text{Mult}(\text{ct}_1, \text{ct}_2)$: For $\text{ct}_i = (b_i, a_i) \in \mathcal{R}_Q^2$ ($i = 0, 1$), compute $(d_0, d_1, d_2) = (b_1 \cdot b_2, a_1 \cdot b_2 + a_2 \cdot b_1, a_1 \cdot a_2)$. Output $\text{ct}_{\text{mult}} \in \mathcal{R}_Q^2$ as

$$\text{ct}_{\text{mult}} \leftarrow (d_0, d_1) + \text{KeySwitch}_{\text{mul}_k}(0, d_2).$$

- $\text{Rot}(\text{ct}, r)$: For $\text{ct} = (b, a) \in \mathcal{R}_Q^2$, output $\text{ct}_{\text{rot}, r} \in \mathcal{R}_Q^2$ as

$$\text{ct}_{\text{rot}, r} \leftarrow (b(X^{5^r}), 0) + \text{KeySwitch}_{\text{rot}_{k_r}}(0, a(X^{5^r})).$$

In particular, we write

$$\text{Rot}(\{\text{ct}_i\}_{i \in I}, r) = \{\text{Rot}(\text{ct}_i, r)\}_{i \in I}$$

for component-wise rotation.

- $\text{Conj}(\text{ct})$: For $\text{ct} = (b, a) \in \mathcal{R}_Q^2$, output $\text{ct}_{\text{conj}} \in \mathcal{R}_Q^2$ as

$$\text{ct}_{\text{conj}} \leftarrow (b(X^{-1}), 0) + \text{KeySwitch}_{\text{cjk}}(0, a(X^{-1})).$$

2.2 Ring-switching Technique

Ring-switching technique [8, 17] switches the underlying ring of HE ciphertexts. Briefly, this method involves applying a key-switching operation to the ciphertext over $\mathcal{R}'$, to obtain a ciphertext with a small subring secret key in $\mathcal{R} \subset \mathcal{R}'$. Recently, [3] highlighted that the ring-switching operation in coefficient-encoded form does not consume any modulus.

For a coefficient-encoded ciphertext $\text{ct} = \text{Enc}_{s'}(m)$ in $\mathcal{R}'_Q$, we switch its secret to s , in which s is a secret in the natural embedding of $\mathcal{R}$ in $\mathcal{R}'$. Then, we obtain $(b, a) \in \mathcal{R}'_Q$ such that

$$b(Y) = -a(Y) \cdot s(Y^2) + m(Y).$$

Once we extract the odd-degree terms of (b, a) , it becomes a ciphertext in $\mathcal{R}_Q^2$ under the secret s , which decrypts to the odd-degree terms of m . Similarly, as we extract the even-degree terms, we yield a ciphertext in $\mathcal{R}_Q^2$ of the even-degree terms of m . It “switches” the underlying ring of ciphertext(s) from $\mathcal{R}'$ to $\mathcal{R}$, and it is called the ring-switching technique. In Section 3.1, we modify the ring-switching technique to switch the ring of “HE keys” rather than HE ciphertexts.

2.3 CKKS FHE Parameters and Key Size

The security of RLWE depends on the maximal modulus, the ring degree, and the distribution of errors and the secret. In this paper, all secret keys and errors are sampled from χ_h of fixed hamming weight h and χ_e with variance $\sigma = 3.19$, respectively. Under this distribution, let $\log Q_{\max, N}$ be the maximum bit-modulus under a ring degree N to guarantee 128-bits of security. Table 2, which is borrowed from [6, Table 3], represents $\log Q_{\max, N}$ for each N and h . From the table, one can check that $N/\log Q_{\max}$ is roughly the same to guarantee the same level of security.

h	$\log Q_{\max}$		
	$\log Q_{\max}(N), \lambda = 128$	$N = 2^{15}$	$N = 2^{16}$
64	$0.015121N - 8.248756$	496	982
96	$0.018896N - 3.671642$	619	1234
128	$0.021370N - 3.601990$	699	1396
192	$0.023448N - 3.611940$	767	1533
$N/2$	[14]	881	1782

Table 2: Relations between bit-size of modulus $\log(Q_{\max})$ and ring degree N for various hamming weight h of secret key s .

Let Q_{key} be the key-switching key modulus and Q_{top} be the maximal ciphertext modulus. Note that Q_{key} should be less than

$Q_{\max}$ for the security, and Q_{top} should be smaller than Q_{key} for the correctness of key-switching. Thus, bigger Q_{key} provides a higher computational capacity. Precisely, $Q_{\text{top}} = Q_{\text{key}}/P$ for a special modulus P . To manage the error growth during key-switching, P should be at least $Q_{\text{top}}^{1/\text{dnum}}$ where dnum is gadget rank for the homomorphic computation. Under fixed $Q_{\max} = Q_{\text{key}}$, we can use larger Q_{top} if using a larger gadget rank dnum .

In FHE, the bootstrapping procedure consists of SlotToCoeff, ModRaise, CoeffToSlot, and EvalMod. The SlotToCoeff and CoeffToSlot steps correspond to a homomorphic Discrete Fourier Transform (DFT) and its inverse, respectively, and both rely on homomorphic rotations [9]. The EvalMod step homomorphically evaluates a polynomial approximation of the mod-1 function, using homomorphic multiplication and conjugation operations. The bootstrapping procedure terminates after EvalMod, and we denote the ciphertext modulus of the refreshed ciphertext by Q_{comp} , which can be used for homomorphic computations.

Assume that we aim to manipulate a ciphertext with a modulus Q and a ring degree of N . The switching key swk belongs to $Q_{\text{key}} = PQ$, where P is a special modulus for key-switching. Note that $P \approx Q^{1/\text{dnum}}$. The key swk consists of dnum ciphertexts in $\mathcal{R}_{N,PQ}^2$, $\{\text{Enc}_s(P\hat{Q}_i \cdot s')\}_{i \in [\text{dnum}]}$, where $\mathbf{G}$ is the gadget vector. Then, to transmit the key, it costs

$$\text{dnum} \cdot N \cdot \log(PQ) \approx (\text{dnum} + 1) \cdot N \cdot \log Q. \quad (1)$$

Notably, the client does not have to send the random 'a' part, because it can be generated by the output of an extendable output-format function (XOF) on a public seed and a counter. This optimization technique is presented in many literatures on lattice-based cryptography including FHE [3, 12].

This observation implies that using a smaller dnum value with a twice larger ring degree might be also an option when focusing on the key size. However, it might degrade the granularity of homomorphic computation.

2.4 Hierarchical Rotation Key System

In [22], the authors introduced a hierarchical rotation key system to reduce the transmission cost. Instead of sending lots of rotation keys, they send a small number of master rotation keys and generate the appropriate rotation keys from them.

The main observation is that a single HE key is a tuple of ciphertexts, which can be manipulated using evaluation keys at higher moduli. The master rotation key is a rotation key at a higher modulus. Suppose one has a master rotation key of r -shift at modulus $P^{(1)}PQ$ and a conventional rotation keys of indices r' at modulus PQ . One generate another rotation key of $(r + r')$ -shift at modulus PQ as follows:

$$\begin{aligned} \text{Rot}(\text{rotk}_{r'}, r) &= \text{Rot}(\{\text{Enc}_s(PG_i \cdot s(X^{5r'}))\}_{i \in [\text{dnum}]}, r) \\ &= \{\text{Enc}_s(PG_i \cdot s((X^{5r'})^{5r}))\}_{i \in [\text{dnum}]} = \text{rotk}_{r+r'}. \end{aligned}$$

Note that a rotation key of index 0 can be generated from public key $\text{pk} = (b, a) = \text{Enc}_s(0)$ as follows:

$$\begin{aligned} \text{rotk}_0 &= \{(b, a + PG_i)\}_{i \in [\text{dnum}]} \\ &= \{\text{Enc}_s(PG_i \cdot s(X))\}_{i \in [\text{dnum}]}. \end{aligned}$$

By combining these two ideas, a client can transmit a small number of *master rotation keys* (e.g., 8 keys), and the server can generate a large number of required rotation keys (e.g., 265 or more) from them. Ideally, given the public key and a master rotation key for a 1-shift, one can generate any rotation key rotk_r by applying $\text{Rot}(\cdot, 1)$ iteratively r times to rotk_0 .

[22] extended the above argument to a multi-level hierarchical system. Note that the generated rotation key can be used as a master rotation key to generate another key in lower moduli. This approach introduces multiple special primes $P = P^{(0)}, P^{(1)}, \dots, P^{(k-1)}$ instead of a single one, within a modulus budget $Q_{\max, N}$ for a fixed ring degree N . They define the ℓ -th level modulus in the chain as $Q^{(\ell)} = Q^{(\ell-1)}P^{(\ell-1)}$, forming a k -level hierarchy $Q = Q^{(0)} < \dots < Q^{(k)} \leq Q_{\max, N}$. Then, a ℓ -th level rotation key of r -shift is

$$\text{rotk}_r^{(\ell)} \leftarrow \text{SwkGen}(N, Q^{(\ell)}, Q^{(\ell)}P^{(\ell)}, s(X^{5r}), s(X)),$$

which has the form:

$$\text{rotk}_r^{(\ell)} = \{\text{Enc}_s(P^{(\ell)}G_i^{(\ell)} \cdot s(X^{5r}))\}_{i \in [\text{dnum}^{(\ell)}]},$$

where $G_i^{(\ell)}$ and $\text{dnum}^{(\ell)}$ are the i -th gadget coefficient and rank for decomposition of $Q^{(\ell)}$, respectively. The k -level hierarchical system of rotation keys is the chain of (master) rotation keys as follows:

$$\{\text{rotk}_r^{(0)}\}_{\mathcal{T}_0} \leftarrow \{\text{rotk}_r^{(1)}\}_{\mathcal{T}_1} \leftarrow \dots \leftarrow \{\text{rotk}_r^{(k-1)}\}_{\mathcal{T}_{k-1}}.$$

Here, $\mathcal{T}_\ell$ is the set of level- ℓ rotation indices. In particular, $\mathcal{T}_{k-1}$ is the set of master keys from the client, and $\mathcal{T}_0$ is the set of target keys for the server. By choosing $\mathcal{T}_0 \supset \mathcal{T}_1 \supset \dots \supset \mathcal{T}_{k-1}$, a small number of master keys can be used to construct all target keys.

We note that under the fixed modulus budget, the level of the hierarchical system gives a trade-off between the key generation efficiency and the homomorphic capacity of the generated keys. A deeper hierarchy (larger k) enables a more efficient key system with fewer transmitted keys; however, it requires a larger $\prod_i P^{(i)}$. In most PPML application parameters, $k = 2$ is an appropriate choice.

In the 2-level setting, [22] sends m master rotation keys (i.e., 1-level keys) of indices from p -ary $\{1, p, p^2, \dots, p^{m-1}\}$, where p is the power-of-two integer closest to $n^{1/m}$ and n is the number of slots. Then all the target 0-level keys of indices can be generated from 1-level keys by approximately $m \cdot p$ rotations of keys.

3 KEY GENERATION OVER EXTENSION RING

For ease of discussion, we distinguish three different types of keys as follows.

- **Transmission key:** is the key that a client sends to the server. It determines the communication cost, and one requires a secret key to generate transmission keys.
- **Evaluation key:** is the key for homomorphic computation. It is the output of key generation steps.
- **Intermediate key:** is the key that appears during key generation steps. Usually, each intermediate key corresponds to an evaluation key.

In this work, the transmission and intermediate keys belong to the extension ring $\mathcal{R}'$, and the evaluation keys belong to a subring $\mathcal{R}$. We note that in [22], the master rotation keys had the role of

the transmission keys. In contrast to ours, their transmission and evaluation keys use the same ring.

In this section, we propose technical tools for generating evaluation keys in $\mathcal{R}$ from transmission keys in $\mathcal{R}'$. Our key generation over $\mathcal{R}'$ consists of two steps:

- (1) generating intermediate keys in $\mathcal{R}'$ from the transmission keys in $\mathcal{R}'$, and
- (2) generating evaluation keys in $\mathcal{R}$ from corresponding intermediate keys in $\mathcal{R}'$.

In particular, to generate a key-switching key from $s_0(X)$ to $s_1(X)$ in $\mathcal{R}$, we consider an intermediate key-switching key from $s_0(Y^2)$ to $\tilde{s}(Y)$ in $\mathcal{R}'$.

In Section 3.2 and 3.3, we describe the first step for rotation keys and general key-switching keys, respectively. Before that, in Section 3.1, we introduce a ring-switching technique for HE keys, which is used for the second step.

3.1 Ring-switching for HE keys

We suggest the ring-switching technique for HE keys, which switches the underlying ring of keys. This is an extension of the ring-switching technique introduced in [8, 17] for HE ciphertexts.

3.1.1 Review of ring-switching for HE ciphertexts. Before introducing the new ring-switching technique for HE keys, we briefly review the ring-switching technique [3, 8, 17] for HE ciphertexts.

Assume that we are given an RLWE-based ciphertext in $\mathcal{R}'^2_Q$ of a plaintext m' in $\mathcal{R}'_Q$. Its secret key is $s'(Y)$ in $\mathcal{R}'$. Then, we switch its secret from s' to another key $s \in \mathcal{R}$ by using switching keys $\text{Enc}_{s(Y^2)}(s'(Y))$, obtaining a ciphertext $(b, a) \in \mathcal{R}'^2_Q$ under the secret s . Let $f(Y) = f_0(X) + Y \cdot f_1(X)$ for $f = a(Y), b(Y)$, and $m(Y)$. Note that f_0 and f_1 belong to $\mathcal{R}_Q$. Then,

$$m_0 + Ym_1 = m \approx as + b = (a_0s + b_0) + Y(a_1s + b_1).$$

Consequently, we obtain two ciphertexts (b_0, a_0) and (b_1, a_1) in $\mathcal{R}_Q^2$ under the secret $s \in \mathcal{R}$, which decrypt to m_0 and m_1 . The overall procedure switches the ring of ciphertext(s) from $\mathcal{R}'$ to $\mathcal{R}$.

3.1.2 Extending ring-switching for HE keys. We modify the ring-switching technique to utilize it for HE keys rather than HE ciphertexts. Technically, we treat a HE key as a tuple of HE ciphertexts and apply the ring-switching technique for each element of the tuple.

We note that a switching key from s' to $\tilde{s}$ for $s', \tilde{s} \in \mathcal{R}'$ takes the form

$$\{\text{Enc}_{s'}(PG_i \cdot \tilde{s})\}_{i \in [\text{dnum}]} \in \mathcal{R}_{PQ}^{2 \times \text{dnum}}$$

where Q is the ciphertext modulus, P is the special modulus for key switching, and dnum is the gadget rank. By using ring-switching from s' to another $s \in \mathcal{R}$ for each ciphertext $\text{Enc}_{s'}(PG_i \cdot \tilde{s})$, one obtains

$$\{\text{Enc}_s(PG_i \cdot \tilde{s}_0)\}_{i \in [\text{dnum}]} \text{ and } \{\text{Enc}_s(PG_i \cdot \tilde{s}_1)\}_{i \in [\text{dnum}]},$$

where $\tilde{s}(Y) = \tilde{s}_0(X) + Y\tilde{s}_1(X)$. This procedure generates two switching keys from s to $\tilde{s}_i$ for $i = 0, 1$. We call the ring-switching key for the keys the “homing key”. Detailed arguments are explained in Theorem 3.1.

THEOREM 3.1 (RING-SWITCHING FOR HE KEYS). *Let $s \in \mathcal{R}$ and $\tilde{s} \in \mathcal{R}'$ be the secret keys and write $\tilde{s}$ as $\tilde{s}(Y) = \tilde{s}_0(X) + Y\tilde{s}_1(X)$ for $\tilde{s}_0, \tilde{s}_1 \in \mathcal{R}$. We define the homing keys hk_0, hk_1 as*

$$\text{hk}_j \leftarrow \text{SwkGen}(N, PQ, Q_{\max, N}, \tilde{s}_j, s),$$

for each $j = 0, 1$. Suppose we have a key-switching key

$$\widetilde{\text{swk}} = \{\text{Enc}_{\tilde{s}}(PG_i \cdot (s'_0 + Ys'_1))\}_{i \in [\text{dnum}]} \in \mathcal{R}'^{2 \times \text{dnum}}_{PQ}$$

with secrets $s'_0, s'_1 \in \mathcal{R}$. Then, one can generate two evaluation keys evk_i (for $i = 0, 1$), each switching a secret key from s'_i to s . This requires $(4 \times \text{dnum})$ key-switching operations over $\mathcal{R}$.

PROOF. Each $(b(Y), a(Y)) = \text{Enc}_{\tilde{s}}(PG_i \cdot (s'_0 + Ys'_1))$ in $\widetilde{\text{swk}}$ satisfies

$$b(Y) + a(Y)\tilde{s}(Y) = PG_i \cdot (s'_0 + Ys'_1) + e(Y) \pmod{PQ}.$$

Putting $b(Y) = b_0 + Yb_1$, $a(Y) = a_0 + Ya_1$, and $e(Y) = e_0 + Ye_1$ for some elements $b_0, b_1, a_0, a_1, e_0, e_1 \in \mathcal{R}_Q$ in the above equation, we obtain the following:

$$\begin{aligned} & (b_0 + a_0\tilde{s}_0 + (Xa_1)\tilde{s}_1) + Y \cdot (b_1 + a_1\tilde{s}_0 + a_0\tilde{s}_1) \\ &= (PG_i \cdot s'_0 + e_0) + Y \cdot (PG_i \cdot s'_1 + e_1) \pmod{PQ}, \end{aligned}$$

since $X = Y^2$. Using the switching key hk_j , which switches secret key from $\tilde{s}_j$ to s , we obtain

$$\begin{aligned} \text{Enc}_s(PG_i \cdot s'_0) &\leftarrow (b_0, 0) + \text{KeySwitch}_{\text{hk}_0}(0, a_0) \\ &\quad + \text{KeySwitch}_{\text{hk}_1}(0, Xa_1), \\ \text{Enc}_s(PG_i \cdot s'_1) &\leftarrow (b_1, 0) + \text{KeySwitch}_{\text{hk}_0}(0, a_1) \\ &\quad + \text{KeySwitch}_{\text{hk}_1}(0, a_0). \end{aligned}$$

Once we conduct the above procedures for all $i \in [\text{dnum}]$, we can obtain the desired two key-switching keys $\text{swk}_0, \text{swk}_1$. $\square$

3.1.3 Choice of secrets and security. In Theorem 3.1, we require two homing keys and two KeySwitch for a single ring-switching. To improve the efficiency of ring-switching, we suggest selecting the secret $\tilde{s}$ in $\mathcal{R}'$ related to the secret s in $\mathcal{R}$.

In particular, we sample the secret keys as follows: we independently sample $s, \tilde{s}_1 \leftarrow \chi_h$ over $\mathcal{R}$, and define $\tilde{s} = s + Y \cdot \tilde{s}_1$ in the extension ring $\mathcal{R}'$. We then use s and $\tilde{s}$ as the HE secret keys for $\mathcal{R}$ and $\mathcal{R}'$, respectively. Since s and $\tilde{s}_1$ are independent, the resulting $\tilde{s}$ in $\mathcal{R}'$ is secure under the MLWE assumption [20] of ring degree N and rank 2, which is at least as hard as RLWE problem over a ring of degree $2N$.

With this choice of secrets, the procedures in Theorem 3.1 can be simplified as

$$\begin{aligned} \text{Enc}_s(PG_i \cdot s'_0) &\leftarrow (b_0, a_0) + \text{KeySwitch}_{\text{hk}}(0, Xa_1), \\ \text{Enc}_s(PG_i \cdot s'_1) &\leftarrow (b_1, a_1) + \text{KeySwitch}_{\text{hk}}(0, a_0), \end{aligned}$$

where $\text{hk} \leftarrow \text{SwkGen}(N, PQ, Q_{\max, N}, \tilde{s}_1, s)$ is a single homing key. Putting everything together, we obtain the following corollary.

COROLLARY 3.2. *From Theorem 3.1, suppose that the secret key $\tilde{s}$ satisfies $\tilde{s}(Y) = s + Y\tilde{s}_1$. If we generate a homing key hk as*

$$\text{hk} \leftarrow \text{SwkGen}(N, PQ, Q_{\max, N}, \tilde{s}_1, s),$$

one can generate two evaluation keys $\text{evk}_0, \text{evk}_1$ from $\widetilde{\text{evk}}$ using $(2 \times \text{dnum})$ key-switching operations in $\mathcal{R}$ with a single homing key hk .

3.2 Key-switching Key Generation over Extension Ring

We describe how to compress two switching keys in $\mathcal{R}$ into a single intermediate key in $\mathcal{R}'$. Assume that we are given a pair of key-switching keys over $\mathcal{R}_{PQ}$ as

$$\text{swk}_{s_j \rightarrow s} = \{\text{Enc}_{\tilde{s}}(PG_i \cdot s_j)\}_{i \in [\text{dnum}]},$$

for each $j = 0, 1$. Now we consider an intermediate key IntSwk over $\mathcal{R}'_{PQ}$ as

$$\text{IntSwk} = \{\text{Enc}_{\tilde{s}}(PG_i \cdot (s_0 + Y \cdot s_1))\}_{i \in [\text{dnum}]},$$

which can be converted into original switching keys $\text{swk}_{s_0 \rightarrow s}$ and $\text{swk}_{s_1 \rightarrow s}$. Now we consider a transmission key of the form:

$$\text{TrmKey} = \text{Enc}_{\tilde{s}}(PP' \cdot (s_0 + Y \cdot s_1)) \in \mathcal{R}'^2_{P'PQ}$$

for another special modulus P' . Our idea is to view TrmKey as a single ciphertext and “multiply it by suitable constants”. Suppose we take the special moduli P' as large as each gadget coefficient G_i . Then, we can multiply TrmKey by each constant G_i/P' , which is small, obtaining the intermediate key IntSwk . Theorem 3.3 describes the correctness in detail.

THEOREM 3.3 (INTERMEDIATE KEY-SWITCHING KEY GENERATION). *Let $\text{swk} = \{\text{Enc}_{\tilde{s}}(PG_i \cdot \tilde{s}')\}_{i \in [\text{dnum}]}$ be a key-switching key underlying in $\mathcal{R}'^{2 \times \text{dnum}}_{PQ}$. If we choose an auxiliary modulus $P' \gtrsim \max_i G_i$, one can generate swk from a single ciphertext $\text{Enc}_{\tilde{s}}(P'P \cdot \tilde{s}') \in \mathcal{R}'^2_{P'PQ}$.*

PROOF. For given ciphertext $(b, a) = \text{Enc}_{\tilde{s}}(P'P \cdot \tilde{s}')$ over $\mathcal{R}'^2_{P'PQ}$, we have

$$b + a\tilde{s} = P'P \cdot \tilde{s}' + e \pmod{P'PQ}.$$

We then multiply by a factor of G_i on both sides to obtain

$$G_i b + G_i a \cdot \tilde{s} = P'PG_i \cdot \tilde{s}' + G_i e \pmod{P'PQ}$$

After rescaling $(G_i b, G_i a)$ by a factor of P' , we obtain

$$\begin{aligned} & \frac{G_i b + \delta_{i,0}}{P'} + \frac{G_i a + \delta_{i,1}}{P'} \cdot \tilde{s} \\ &= PG_i \cdot \tilde{s}' + \frac{G_i e_i}{P'} + e_{rs} \pmod{PQ}, \end{aligned}$$

where $e_{rs} = (\delta_{i,0} + \delta_{i,1} \cdot \tilde{s})/P'$ with small coefficients polynomial $\delta_{i,0}/P', \delta_{i,1}/P'$. Then, it follows that the error term $G_i e/P' + e_{rs}$ becomes sufficiently small, and we obtain the desired rotation key over $\mathcal{R}'^2_{PQ}$ from the following procedure

$$\text{Enc}_{\tilde{s}}(PG_i \cdot \tilde{s}') \leftarrow \left(\frac{G_i b + \delta_{i,0}}{P'}, \frac{G_i a + \delta_{i,1}}{P'} \right)$$

for each i . $\square$

To put Corollary 3.2 and Theorem 3.3 together, one can generate two switching keys from a single compact transmission key TrmKey . Specifically, the intermediate key IntSwk is first generated from TrmKey using Theorem 3.3, and then decomposed into two switching keys using Corollary 3.2. For example, if a client

wants to transmit mul and cjk for multiplication and conjugation, respectively, it suffices to send

$$\text{TrmEvk} = \text{Enc}_{\tilde{s}}(PP' \cdot (s(X)^2 + Y \cdot s(X^{-1})))$$

over $\mathcal{R}'_{P'PQ}$ with a homing key hk to a server. Then, the server generates the required mul and cjk via the following intermediate key:

$$\text{IntEvk} = \{\text{Enc}_{\tilde{s}}(PG_i \cdot (s^2(X) + Y \cdot s(X^{-1})))\}_{i \in [\text{dnum}]},$$

Notably, once we choose $P' \gtrsim \max_i \{Q/Q_i\} \simeq Q/Q^{1/\text{dnum}}$, then it follows that $P'PQ \simeq Q^2 < Q_{\max,N}^2 \simeq Q_{\max,2N}$, ensuring that the transmission key $\text{TrmEvk} \in \mathcal{R}'^2_{P'PQ}$ does not exceed the security margin.

3.3 Rotation Key Generation over Extension Ring

A rotation key is one of the key-switching keys, which can be generated more efficiently than others such as mul and cjk . As a naive approach, one may apply the techniques in Sections 3.1 and 3.2 to reduce the communication cost of sending rotation keys. However, since many rotation keys are often required (e.g., 265 or more for ResNet-20 inference [22]), the total communication cost may still be unsatisfactory.

[22] proposed a remarkable solution to this problem by adopting a hierarchical key management system (see Section 2.4). Yet, their approach remains limited. A conventional choice for FHE parameters is to use a ring degree $N = 2^{16}$, with Q_{key} set close to $Q_{\max,N}$ to fully utilize the top-level ciphertext modulus Q_{top} . This tight budget makes it difficult to incorporate multiple special moduli $p^{(0)}, \dots, p^{(k-1)}$, thus restricting the hierarchy to $k = 2$ in practice.

In a 2-level hierarchy with $Q^{(2)} = p^{(1)}Q_{\text{key}} = Q_{\max,N}$, the master rotation keys become large due to the increased gadget rank $\text{dnum}^{(1)} = \lceil \log Q_{\text{key}} / \log p^{(1)} \rceil$ for small $p^{(1)}$. Moreover, sending too few master rotation keys leads to inefficient server-side key generation. In practice, [22] transmits 8 master rotation keys with a 16-ary index set, which still results in several gigabytes of key size.

Hierarchical key system with an extension ring. To overcome these limitations, we propose a new strategy that generates intermediate rotation keys in the extension ring $\mathcal{R}'$ instead of $\mathcal{R}$. This approach benefits from a larger modulus budget $Q_{\max,2N}$, roughly doubling the bit-length available compared to $\mathcal{R}$. This extra budget allows us to construct a higher level hierarchy with a very small top-level gadget rank, say $\text{dnum} = 1$, compared to the hierarchy over $\mathcal{R}$.

Suppose we want to generate rotation keys for an index set $\mathcal{T}_0$ under a modulus $Q^{(1)} = PQ$ that is strictly smaller than $Q_{\max,N}$. Then, since

$$\log Q_{\max,2N} \approx 2 \log Q_{\max,N} > \log Q_{\max,N} > \log Q^{(1)},$$

there is sufficient room in the modulus budget to fully leverage a deeper hierarchy. In particular, one can construct a 3-level hierarchy with

$$Q^{(2)} = Q_{\max,N} \text{ and } Q^{(3)} = Q_{\max,2N}.$$

This allows the top gadget rank to be set as $\text{dnum}^{(2)} = 1^2$, so that each master rotation key becomes small. In comparison to the 2-level case, we observe that

$$2N \cdot \log Q_{\max, 2N} < \text{dnum}^{(1)} \cdot N \cdot \log Q_{\max, N}$$

if $\text{dnum}^{(1)} > 4$, where the left-hand side and right-hand side correspond to the size of each master rotation key in the 3-level system over $\mathcal{R}'$ and the 2-level system over $\mathcal{R}$, respectively. Since $\text{dnum}^{(1)} = \lceil \log Q^{(1)} / \log P^{(1)} \rceil$ is typically much bigger than 4 under the conventional parameter setting for $\mathcal{R}$, our construction enjoys significantly smaller master rotation keys.

Moreover, this 3-level hierarchical system allows the client to transmit fewer master rotation keys than in the 2-level hierarchy. For example, assume that we are given a 2-level hierarchical system with the transmission keys from the indices set $\mathcal{T}_1$. One may consider a 3-level system with the transmission keys from the indices set $\mathcal{T}_2$ that is a subset of $\mathcal{T}_1$. Then, one can generate intermediate keys with indices of $\mathcal{T}_1$ at level 1 and follow the previous 1-level key generation procedure with the intermediate keys. Note that in practical implementation, one can consider various indices sets for the intermediate keys to improve the key generation efficiency. We refer to Section 5 for the practical examples of the choice of indices.

Throughout our new hierarchical rotation key system, we generate a set of intermediate rotation keys $\{\text{rotk}_r'^{(0)}\}_{r \in \mathcal{T}_0}$, each of the form

$$\text{rotk}_r'^{(0)} = \{\text{Enc}_{\tilde{s}}(PG_i \cdot \tilde{s}(Y^{5^r}))\}_{i \in [\text{dnum}]} \in \mathcal{R}_{PQ}^{2 \times \text{dnum}},$$

which serves as a rotation key over $\mathcal{R}'$. Theorem 3.4, an analogy of Theorem 3.1 for intermediate rotation keys, guarantees that the same set of rotation keys $\mathcal{T}_0$ can be correctly recovered over $\mathcal{R}$ via ring-switching.

THEOREM 3.4 (INTERMEDIATE ROTATION KEY GENERATION). *Let $s(X) \in \mathcal{R}$ and $\tilde{s}(Y) = s + Y\tilde{s}_1 \in \mathcal{R}'$ be secret keys, and let hk be the homing key from Corollary 3.2. If we apply ring-switching to a rotation key of index r over $\mathcal{R}'$ using hk , we obtain a rotation key of the same index r over $\mathcal{R}$.*

PROOF. For a ciphertext $(b(Y), a(Y)) = \text{Enc}_{\tilde{s}}(PG_i \cdot \tilde{s}(Y^{5^r}))$ over $\mathcal{R}'_{PQ}$, we have

$$b(Y) + a(Y) \cdot \tilde{s}(Y) = PG_i \cdot \tilde{s}(Y^{5^r}) + e(Y) \pmod{PQ}.$$

From $\tilde{s}(Y) = s_0(Y^2) + Y \cdot \tilde{s}_1(Y^2)$, we have

$$\begin{aligned} \tilde{s}(Y^{5^r}) &= s(Y^{2 \cdot 5^r}) + Y^{5^r} \cdot \tilde{s}_1(Y^{2 \cdot 5^r}) \\ &= s(X^{5^r}) + Y \cdot (X^{\frac{5^r-1}{2}} \tilde{s}_1(X^{5^r})). \end{aligned}$$

By the same arguments as in Theorem 3.1, once we put $b(Y) = b_0 + Yb_1$, $a(Y) = a_0 + Ya_1$, and $e(Y) = e_0 + Ye_1$, we obtain one of the equation with respect to X as follows:

$$b_0 + a_0s + (X \cdot a_1)\tilde{s} = PG_i \cdot s(X^{5^r}) + e_0 \pmod{PQ}.$$

Therefore, it follows that

$$\text{Enc}_s(PG_i \cdot s(X^{5^r})) \leftarrow (b_0, a_0) + \text{KeySwitch}_{\text{hk}}(0, Xa_1),$$

which becomes i -th ciphertext of the rotation key of the index r over $\mathcal{R}$. $\square$

²More generally, one can set $Q^{(3)} \approx Q_{\max, 2N}$ and $Q^{(2)} \approx \frac{d}{d+1} Q_{\max, 2N}$ to use a small $\text{dnum}^{(2)} = d$.

4 KEY MANAGEMENT SYSTEMS

In this section, we present our new key management systems, KG^+ and BTS^+ . We describe the structure of the transmission keys in each system and report the total transmission key sizes. As noted at the beginning of Section 3, we distinguish between two types of keys: transmission keys, which are sent from the client to the server, and evaluation keys, which are used for homomorphic computation. The transmission key size determines the communication cost between the client and the server. Throughout this section, we follow the notation and conventions established in Sections 2 and 3.

4.1 KG^+ Key Management System

We first propose a new key management system, KG^+ . A server requires an encryption key enck over $\mathcal{R}_Q$, along with evaluation keys over $\mathcal{R}_{PQ}$: namely, a multiplication key mul , a conjugation key cjk , and rotation keys $\{\text{rotk}_r\}_{r \in \mathcal{T}_0}$ with index set $\mathcal{T}_0$.

To enable this, a client generates two secret keys $s, s' \in \mathcal{R}$ and defines an extended key $\tilde{s}(Y) = s + Ys' \in \mathcal{R}'$. Then, KG^+ constructs a k -level hierarchy of rotation keys with respect to $\tilde{s}$ over $\mathcal{R}'$, typically with $k = 3$, and the client sends the following transmission keys:

- Encryption key $\text{enck}' = \text{Enc}_{\tilde{s}}(0)$ over $\mathcal{R}'_{Q^{(k-1)}}$,
- Transmission key over $\mathcal{R}'_{P'PQ}$ as

$$\text{TrmEvk} = \text{Enc}_{\tilde{s}}(P'P(s^2(X) + Ys(X^{-1}))),$$

- Master rotation keys $\{\text{rotk}_r'^{(k-1)}\}_{r \in \mathcal{T}_{k-1}}$ over $\mathcal{R}'_{Q^{(k)}}$ with $m = |\mathcal{T}_{k-1}|$,
- Homing key over $\mathcal{R}_{Q_{\max, N}}$ as

$$\text{hk} \leftarrow \text{SwkGen}(N, PQ, Q_{\max, N}, s', s).$$

Using Corollary 3.2, the server can generate an encryption key enck from received enck' and hk . Combining Corollary 3.2 with Theorem 3.3, the server can also generate mul and cjk from the received TrmEvk and hk . Furthermore, a k -level hierarchical system allows the server to generate the intermediate rotation keys $\{\text{rotk}_r'^{(0)}\}_{r \in \mathcal{T}_0}$ for the required index set $\mathcal{T}_0$, using master rotation keys and enck' . The final rotation keys $\{\text{rotk}_r\}_{r \in \mathcal{T}_0}$ over $\mathcal{R}$ are then obtained by applying Theorem 3.4. Notably, the server does not use $\mathcal{R}'$ after key generation, resulting in no degradation in the latency of homomorphic computation.

Transmission key size. Each transmission key size is estimated as follows:

- enck' : $2N \cdot \log Q^{(k-1)}$,
- TrmEvk : $2N \cdot \log P'PQ \approx 2N \cdot 2 \log Q$,
- master rotation keys: $|\mathcal{T}_{k-1}| \cdot \text{dnum}^{(k-1)} \cdot 2N \cdot \log Q^{(k)}$,
- hk : $\text{dnum}^{(h)} \cdot N \cdot \log Q_{\max, N}$.

Here, we set $P \approx Q^{1/\text{dnum}}$ and $P' \approx Q/Q^{1/\text{dnum}}$, and let $\text{dnum}^{(h)}$ denote the gadget rank for modulus QP required for key-switching using homing key hk at modulus $Q_{\max, N}$, i.e.,

$$\text{dnum}^{(h)} = \left\lceil \frac{\log PQ}{\log Q_{\max, N} - \log PQ} \right\rceil.$$

To put it all together, the overall size of the transmission keys is estimated as:

$$(2 \log Q^{(k-1)} + 4 \log Q + m \cdot \text{dnum}^{(k-1)} \cdot 2 \log Q^{(k)} + \text{dnum}^{(h)} \log Q_{\max, N}) \cdot N.$$

4.2 BTS⁺ Key Management System

In KG⁺, the transmission key size is effectively reduced. However, the size of homing key hk still remains unsatisfactory, as it relies on $\mathcal{R}$ and requires a large gadget rank $\text{dnum}^{(h)}$ due to the limited modulus budget—namely, the modulus PQ is already close to $Q_{\max, N}$.

We describe a modified key management system, BTS⁺, which can reduce the size of the homing key to the desired amount specified by the server, albeit with a loss in computation efficiency.

To reduce the homing key size, we select a small $\text{dnum}^{(h)}$ in BTS⁺. This small-sized homing key allows the ring-switching for the keys of which its modulus is under Q_{comp} . Then, the computations over $\mathcal{R}$ are restricted to ciphertexts after bootstrapping, which have modulus Q_{comp} . The bootstrapping procedures for ciphertexts with modulus $Q > Q_{\text{comp}}$ —namely, *CoeffToSlot* and *EvalMod*—are performed over $\mathcal{R}'$ instead of $\mathcal{R}$. Concretely, we adopt the additional modulus chain over $\mathcal{R}$ as:

$$Q_{\text{comp}} < Q^{(h)} = P_{\text{comp}} Q_{\text{comp}} < Q_{\max, N} = P^{(h)} Q^{(h)}.$$

Here, $Q^{(h)}$ is the modulus that homing is performed. It serves as the new key-switching modulus over $\mathcal{R}$, and the gadget rank $\text{dnum}^{(h)}$ for hk to apply ring-switching for these keys is given by

$$\text{dnum}^{(h)} = \left\lceil \frac{\log Q^{(h)}}{\log Q_{\max, N} - \log Q^{(h)}} \right\rceil. \quad (2)$$

In our BTS⁺ estimations, we used the parameters that satisfy $\text{dnum}^{(h)}$ is either 1 or 2. This significantly reduces the homing key size compared to KG⁺.

It is important that the small $\text{dnum}^{(h)}$ does not degrade the throughput of homomorphic computation after the key generation. For conventional CKKS parameters [5, 6] with the ring degree less than 2^{17} , we can generate rotation keys with gadget rank 2 in $\mathcal{R}$ from the homing key with $\text{dnum}^{(h)}$ either 1 or 2. Note that 2 is the computation-optimal gadget value for fixed ciphertext modulus [24]. For the homomorphic computation over $\mathcal{R}'$, our system mostly perform FHE bootstrapping, which is a highly parallelizable operation. One can efficiently perform the bootstrapping over $\mathcal{R}'$ with similar throughput to $\mathcal{R}$.

We now describe the transmission keys for BTS⁺, following the same setup in KG⁺. Here, a server requires an encryption key enc over $\mathcal{R}_{Q_{\text{comp}}}$, evaluation keys over $\mathcal{R}_{Q^{(h)}}$, and evaluation keys for bootstrapping over $\mathcal{R}'_{PQ}$. A client in BTS⁺ sends the following transmission keys:

- Encryption key $enc' = \text{Enc}_{\tilde{s}}(0)$ over $\mathcal{R}'_{Q^{(k-1)}}$,
- Transmission keys over $\mathcal{R}'_{P'PQ}$ as

$$\text{TrmMulK} = \text{Enc}_{\tilde{s}}(P'P \cdot \tilde{s}^2(Y)), \text{ and}$$

$$\text{TrmCjk} = \text{Enc}_{\tilde{s}}(P'P \cdot \tilde{s}(Y^{-1})),$$

- Transmission key over $\mathcal{R}'_{P'_{\text{comp}}Q^{(h)}}$ as

$$\text{TrmEvk} = \text{Enc}_{\tilde{s}}(P'_{\text{comp}}P_{\text{comp}}(s^2(X) + Ys(X^{-1}))),$$

- Master rotation keys $\{\text{rotk}'_r\}_{r \in \mathcal{T}_{k-1}^{(k-1)}}$ over $\mathcal{R}'_{Q^{(k)}}$ with $m = |\mathcal{T}_{k-1}|$,
- Homing key over $\mathcal{R}_{Q_{\max, N}}$ as

$$hk \leftarrow \text{SwkGen}(N, Q^{(h)}, Q_{\max, N}, s', s),$$

- Leaving key over $\mathcal{R}'_{pq}$ as

$$lk \leftarrow \text{SwkGen}(2N, q, qp, s, \tilde{s}),$$

The server generates mulK' and cjk' in $\mathcal{R}'_{PQ}$ from TrmMulK and TrmCjk using Theorem 3.3, while mulK and cjk in $\mathcal{R}_{Q^{(h)}}$ are generated from a single TrmKey . The server generates rotation keys for both $\mathcal{R}$ and $\mathcal{R}'$, i.e., $\{\text{rotk}_r\}_{r \in \mathcal{T}_0}$ and $\{\text{rotk}'_r\}_{r \in \mathcal{T}_{\text{BTS}}}$, respectively. Here, the leaving key lk supports ring-switching for ciphertexts at base modulus q , enabling their transition from $\mathcal{R}$ to $\mathcal{R}'$, where bootstrapping is performed. Both p and $q \gtrsim p$ are small primes chosen to set the gadget rank of the leaving key lk to 1.

Transmission key size. We describe the size of each transmission keys in BTS⁺. The following keys are newly introduced or updated in BTS⁺:

- $\text{TrmMulK}, \text{TrmCjk}$: $4N \cdot \log P'PQ \approx 4N \cdot 2 \log Q$,
- TrmEvk : $2N \cdot \log P'_{\text{comp}}Q^{(h)} \leq 2N \cdot \log Q_{\max, N}$,
- hk : $\text{dnum}^{(h)} \cdot N \cdot \log Q_{\max, N} \leq 2N \cdot \log Q_{\max, N}$.
- lk : $2N \cdot \log pq \approx 4N \cdot \log q$.

The advantage of reducing hk size from $\text{dnum}^{(h)}$ to 1 or 2 more than compensates for the overhead introduced by the additional keys. To put it all together, the overall size of the transmission keys is estimated to be at most:

$$(2 \log Q^{(k-1)} + 8 \log Q + m \cdot \text{dnum}^{(k-1)} \cdot 2 \log Q^{(k)} + 4 \log q + 4 \log Q_{\max, N}) \cdot N. \quad (3)$$

5 IMPLEMENTATION

We implemented KG⁺ and [22] by using the HEaAN CKKS library [13]. The client-side experiments were measured on an Intel Xeon gold 6542Y with a single-thread CPU. The server-side experiments were measured on NVIDIA A100 80GB PCIe GPU.

5.1 Parameter Settings

For the fair comparison to [22], we use the same rings and generate the same evaluation keys as in [22]. The full details are reported in Tables 3 and Table 4 (A.i, A.ii, and A.iii). Notably, we adopt the parameter sets with ring degree 2^{16} from [22], which is the conventional choice in most implementations. All parameters are 128-bit secure according to [22, 25].

Table 3: The CKKS parameters of ring $\mathcal{R}$ and extension ring $\mathcal{R}'$.

	Ring $\mathcal{R}$	Ext. ring $\mathcal{R}'$
ring degree N	2^{16}	2^{17}
secret key hamming weight h	2^{15}	2^{16}
SD of gaussian error σ	3.2	3.2
$\log Q_{\max}$	1714	3428

For each parameter set in A, we use the corresponding B and E parameter sets to implement the two-level hierarchical systems of [22] and KG^+ . For the three-level systems, we used C and D parameter sets for KG^+ and F parameter set for BTS^+ . Note that we use ring $\mathcal{R}$ for A and B parameter sets, and the extension ring $\mathcal{R}'$ for C, D, E, and F parameter sets. For each of the KG^+ parameter sets C, D, and E, the ring-switching procedure and the homing keys follow the corresponding B parameter sets. In contrast, ring switching and the homing keys for the BTS^+ are derived from the F parameter set. Further details are provided in Table 4.

Table 4: Parameter sets in our KG^+ implementations. We use the original 1-level A and 2-level B parameter sets from [22] over the ring $\mathcal{R}$. We then construct new C, D, and E parameter sets for KG^+ , where the moduli are derived from the original 1-level parameters. Here, $dnum$ and Modulus bit length denote the tuples $(dnum^{(0)}, \dots, dnum^{(k-1)})$ and $(\log Q^{(0)}, \log P^{(0)}, \dots, \log P^{(k-1)})$ for a k -level hierarchy, respectively.

Parameters		$dnum$	Modulus bit length
Original (1-level)	A.i	4	(1321, 333)
	A.ii	5	(1321, 273)
	A.iii	6	(1321, 221)
[22] (2-level)	B.i	(4, 30)	(1321, 333, 59)
	B.ii	(5, 15)	(1321, 273, 118)
	B.iii	(6, 10)	(1321, 221, 171)
KG^+ (3-level)	C.i	(4, 30, 1)	(1321, 333, 59, 1715)
	C.ii	(5, 15, 1)	(1321, 273, 118, 1715)
	C.iii	(6, 10, 1)	(1321, 221, 171, 1715)
	D.i	(4, 3, 2)	(1321, 333, 560, 1140)
	D.ii	(5, 3, 2)	(1321, 273, 560, 1140)
KG^+ (2-level)	D.iii	(6, 3, 2)	(1321, 221, 560, 1083)
	E.i	(4, 1)	(1321, 333, 1668)
	E.ii	(5, 1)	(1321, 273, 1660)
BTS^+ (3-level)	E.iii	(6, 1)	(1321, 221, 1578)
	F.i	(2, 2, 1)	(661, 385, 667, 1715)
	F.ii	(2, 2, 1)	(673, 432, 607, 1715)
	F.iii	(2, 2, 1)	(661, 385, 667, 1715)

For [22], we use the B-parameter sets. For the experiment related to ResNet-20, it sends master rotation keys corresponding to 16-ary indices $\{\pm 1, \pm 16, \pm 16^2, \pm 16^3\}$, including both positive and negative values. For the experiment related to bootstrapping, it sends master rotation keys corresponding to 32-ary indices $\{\pm 1, \pm 2^5, \pm 2^{10}\}$.

For KG^+ , we generate the evaluation keys with B-parameter sets by using transmission keys with the C, D, and E parameter sets. In particular, after hierarchical key generation over $\mathcal{R}'$ using the C, D, and E parameter sets, we switch the parameters of each generated intermediate key $rotk_r'$ into B-parameter set by using the homing. It turns into the desired rotation key $rotk_r$ over $\mathcal{R}$. For BTS^+ , from given transmission keys with F parameter, we generate two evaluation key sets; one with F parameter for general homomorphic computation in $\mathcal{R}$ and the other with C parameter for bootstrappings in $\mathcal{R}'$.

When we use C parameter set for KG^+ , to optimize the size of the master rotation keys, we send 2 master rotation keys of rotation indices $\{1, 2^8\}$. During the key generation, we generate 1-level rotation keys with indices from the 4-ary set $\{1, 2^2, 2^4, 2^6, 2^8, 2^{10}, 2^{12}, 2^{14}\}$. The final rotation keys at 0-level are generated from these 1-level

Table 5: Experimental results for generating evaluation keys for ResNet-20 with the CKKS parameters A.i, A.ii, and A.iii. Key size is the total transmission key size in GB. Client/server runtime is the key generation time in seconds.

Base	Parameters		Key Size	Client Runtime	Server Runtime
A.i	Original	A.i	14.5	61.6	-
	[22]	B.i	3.49	14.1	45.7
	KG ⁺	C.i	0.609	0.99	107
		D.i	0.725	1.99	57.8
	BTS ⁺	F.i	0.285	1.09	23.52
A.ii	Original	A.ii	17.4	73.0	-
	[22]	B.ii	1.82	7.83	49.9
	KG ⁺	C.ii	0.397	1.02	96.3
		D.ii	0.508	2.02	74.5
	BTS ⁺	F.ii	0.285	1.06	26.61
A.iii	Original	A.iii	20.2	80.8	-
	[22]	B.iii	1.29	5.48	46.6
	KG ⁺	C.iii	0.325	1.00	97.8
		D.iii	0.397	1.02	96.3
	BTS ⁺	F.iii	0.285	1.08	23.42

4-ary keys. This parameter provides good client-side costs with a reasonable key generation time.

For the D parameter set, we do almost the same as for the C parameter set. Additionally, to further accelerate the server-side key generation, we set the $dnum$ value of the master rotation key ($dnum^{(2)}$) as 2, and we adjust the other $dnum$ values accordingly.

For the E parameter set, we use 2-level KG^+ with 3 master rotation keys. It results in a more efficient key generation process. This is particularly relevant for FHE parameter settings where only a small number of rotation keys are needed to support bootstrapping. For E parameter, we choose the master rotation key indices as $\{1, 2^5, 2^{10}\}$.

We use the F parameter set to implement BTS^+ for the A parameter set. To make the A parameter set bootstrappable, we allocate 3 levels each to SlotToCoeff and CoeffToSlot, and 9 levels to EvalMod. After excluding the levels required for CoeffToSlot and EvalMod, the remaining levels are allocated to Q_{comp} for homomorphic computations and SlotToCoeff, as in all A parameter sets. We generate rotation keys for CoeffToSlot, as well as a multiplication key and a conjugation key for EvalMod over $\mathcal{R}'$. All the other evaluation keys are generated over $\mathcal{R}$. Here, we use the same master evaluation key with rotation indices $\{1, 2^8\}$ and the same 1-level intermediate keys with indices $\{1, 2^2, 2^4, 2^6, 2^8, 2^{10}, 2^{12}, 2^{14}\}$ in the key generation procedures for both $\mathcal{R}$ and $\mathcal{R}'$.

5.2 Key Generation for ResNet-20 Models

We present the experimental results of [22] and KG^+ for generating evaluation keys, including the 265 rotation keys required for ResNet-20 inference on the CIFAR-10 dataset, as considered in prior work [21]. We refer to [22] for the exact index sets of required 265 rotation keys for ResNet-20 inference. Here, the target rotation keys lie in the ring $\mathcal{R}$, where the number of slots is $n = 2^{15}$. Experimental results for all configurations are presented in Table 5.

We note that KG^+ with the C parameter sets reduces the key size by $3.95\text{--}5.73\times$ and client-side runtime by $5.48\text{--}14.25\times$ compared to [22]; while it increases the server-side runtime within the factors of $1.95\text{--}2.35$. The D parameter sets increase the key size and the client-side runtime compared to C parameter sets, but it lowers the server-side runtime by factors of $1.26\text{--}1.74$. We note that all KG^+ experimental results show substantially improved client-side costs. In addition, we report the estimated transmission key size of BTS^+ for the ResNet-20 task in Table 6.

For BTS^+ , we generate 265 rotation keys, 1 multiplication key, and 1 conjugation key over $\mathcal{R}$ based on the F parameter set with ring-switching. This process matches KG^+ , but uses a much smaller $dnum$ with $dnum^{(h)} = 2$ for ring-switching. The smaller $dnum^{(h)}$ benefits BTS^+ in reducing the key size by $4.53\text{--}12.25\times$, client-side runtime by $5.07\text{--}12.94\times$, and server-side runtime by $1.88\text{--}1.99\times$ compared to [22]. Note that we additionally generate 38 rotation keys³ for CoeffToSlot, 1 multiplication key, and 1 conjugation key for EvalMod over $\mathcal{R}'$, based on C parameter set without using ring-switching.

5.3 Key Generation for Bootstrapping

We present the experimental results for generating evaluation keys for the CKKS scheme with a reasonable bootstrapping in a ring of degree 2^{16} . We assume that during the bootstrapping, the server leverages a multiplication key, a conjugation key, and 38 rotation keys. This is a common configuration for FHE parameters in many CKKS libraries, including [1, 2, 13]. The experimental results are reported in Table 6.

We first note that [22] does not significantly reduce the key size or client runtime. This is because the server needs a relatively smaller number of rotation keys in this setting. As each master rotation key in [22] is larger than each evaluation key, the total key size for [22] is not much smaller than the size of 38 evaluation keys. In particular, for parameter set A.i, it is even bigger than 38 rotation keys.

On the other hand, KG^+ with the C parameter set reduces the key size by $3.09\text{--}4.37\times$ and client-side runtime by $5.55\text{--}10.65\times$ compared to [22]. It increases the server-side runtime within the factors of $2.28\text{--}4.78$. The E parameter sets slightly increase size and client runtime compared to C parameter sets, but it lowers the server-side runtime by the factors of $1.53\text{--}1.58$. All KG^+ experiments show substantially better client-side costs than [22].

ACKNOWLEDGMENTS

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2025-02217656). We also acknowledge that part of this work was done while the third author was at Seoul National University.

REFERENCES

- [1] 2024. Lattigo v6. EPFL-LDS, Tune Insight SA, Available at <https://github.com/tuneinsight/lattigo>.

³Precisely, the indices are: $\mathcal{T}_{bts} = \{1, 2, 3, 4, 5, 6, 7, 8, 16, 24, 32, 64, 96, 128, 160, 192, 224, 256, 512, 768, 1024, 2048, 3072, 4096, 5120, 6144, 7168, 8192, 16384, 24576, 31744, 32000, 32256, 32512, 32736, 32744, 32752, 32760\}$.

Table 6: Experimental results for generating evaluation keys for the bootstrapping for CKKS parameters A.i, A.ii, and A.iii. Key size is the total transmission key size in GB. Client/server runtime is the key generation time in seconds.

Base	Parameters		Key Size	Client Runtime	Server Runtime
A.i	Original	A.i	2.18	12.3	-
	[22]	B.i	2.65	10.6	8.55
	KG^+	C.i	0.609	0.99	40.9
		E.i	0.659	1.26	13.1
A.ii	Original	A.ii	2.62	14.2	-
	[22]	B.ii	1.40	6.25	8.06
	KG^+	C.ii	0.397	1.02	32.5
		E.ii	0.443	1.28	13.3
A.iii	Original	A.iii	3.04	15.9	-
	[22]	B.iii	1.00	5.55	8.97
	KG^+	C.iii	0.325	1.00	20.4
		E.iii	0.364	1.20	14.2

- [2] Ahmad Al Badawi, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Ian Quah, Yuriy Polyakov, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Suponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. 2022. OpenFHE: Open-Source Fully Homomorphic Encryption Library. In *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography* (Los Angeles, CA, USA) (WAHC'22). Association for Computing Machinery, New York, NY, USA, 53–63. <https://doi.org/10.1145/3560827.3563379>
- [3] Y. Bae, J. H. Cheon, J. Kim, J. H. Park, and D. Stehlé. 2023. HERMES: Efficient Ring Packing Using MLWE Ciphertexts and Application to Transciphering. In *CRYPTO*. https://doi.org/10.1007/978-3-031-38551-3_2
- [4] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. 2024. Bootstrapping bits with CKKS. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 94–123. https://doi.org/10.1007/978-3-031-58723-8_4
- [5] Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, et al. 2024. Security guidelines for implementing homomorphic encryption. (2024). Available at <http://eprint.iacr.org/2024/463>.
- [6] Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *EUROCRYPT*. Springer, 587–617. https://doi.org/10.1007/978-3-030-77870-5_21
- [7] Z. Brakerski. 2012. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *CRYPTO*. https://doi.org/10.1007/978-3-642-32009-5_50
- [8] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)* 6, 3 (2014), 1–36. <https://doi.org/10.1145/2633600>
- [9] Jung Hee Cheon, Kyoohyun Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for approximate homomorphic encryption. In *EUROCRYPT*. Springer, 360–384. https://doi.org/10.1007/978-3-319-78381-9_14
- [10] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*. Springer, 409–437. https://doi.org/10.1007/978-3-319-70694-8_15
- [11] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. 2017. Faster Packed Homomorphic Operations and Efficient Circuit Bootstrapping for TFHE. In *ASIACRYPT*. https://doi.org/10.1007/978-3-319-70694-8_14
- [12] Jean-Sébastien Coron, Avradip Mandal, David Naccache, and Mehdi Tibouchi. 2011. Fully homomorphic encryption over the integers with shorter public keys. In *CRYPTO*. https://doi.org/10.1007/978-3-642-22792-9_28
- [13] CryptoLab. 2022. HEaAN Library. Available at <https://heaan.it/>.
- [14] Benjamin R Curtis and Rachel Player. 2019. On the feasibility and impact of standardising sparse-secret LWE parameter sets for homomorphic encryption. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. 1–10. <https://doi.org/10.1145/3338469.3358940>
- [15] L. Ducas and D. Micciancio. 2015. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *EUROCRYPT*. https://doi.org/10.1007/978-3-662-46800-5_24

- [16] J. Fan and F. Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. Available at <http://eprint.iacr.org/2012/144>.
- [17] Craig Gentry, Shai Halevi, Chris Peikert, and Nigel P Smart. 2013. Field switching in BGV-style homomorphic encryption. *Journal of Computer Security* 21, 5 (2013), 663–684. <https://doi.org/10.3233/JCS-130480>
- [18] Shai Halevi and Victor Shoup. 2020. Design and implementation of HELib: a homomorphic encryption library. (2020). Available at <https://github.com/homenc/HElib>.
- [19] Kyoohyung Han and Dohyeong Ki. 2020. Better bootstrapping for approximate homomorphic encryption. In *Cryptographers' Track at the RSA Conference*. Springer, 364–390. https://doi.org/10.1007/978-3-030-40186-3_16
- [20] Adeline Langlois and Damien Stehlé. 2015. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography* 75, 3 (2015), 565–599. <https://doi.org/10.1007/s10623-014-9938-4>
- [21] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. 2022. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In *International Conference on Machine Learning*. PMLR, 12403–12422. <https://proceedings.mlr.press/v162/lee22e.html>
- [22] Joon-Woo Lee, Eunsang Lee, Young-Sik Kim, and Jong-Seon No. 2023. Rotation Key Reduction for Client-Server Systems of Deep Neural Network on Fully Homomorphic Encryption. In *ASIACRYPT*. Springer, 36–68. https://doi.org/10.1007/978-981-99-8736-8_2
- [23] V. Lyubashevsky, C. Peikert, and O. Regev. 2010. On Ideal Lattices and Learning with Errors over Rings. In *EUROCRYPT*. https://doi.org/10.1007/978-3-642-13190-5_1
- [24] Johannes Mono and Tim Güneysu. 2023. A new perspective on key switching for BGV-like schemes. (2023). Available at <http://eprint.iacr.org/2023/1642>.
- [25] Johannes Mono, Chiara Marcolla, Georg Land, Tim Güneysu, and Najwa Aaraj. 2023. Finding and evaluating parameters for BGV. In *International Conference on Cryptology in Africa*. Springer, 370–394. https://doi.org/10.1007/978-3-031-37679-5_16
- [26] D. Stehlé, R. Steinfeld, K. Tanaka, and K. Xagawa. 2009. Efficient Public Key Encryption Based on Ideal Lattices. In *ASIACRYPT*. https://doi.org/10.1007/978-3-642-10366-7_36